

1. Briefly describe the artifact.

The original artifact was the portfolio item for CS-300 completed in December of 2025. It was written in C++ and features a CSV parser written by me. This parser creates a 2D vector of academic courses with prerequisites that can be searched efficiently using a binary search tree with various algorithms.

CSV PARSER

```
def csv_parser(file_path): 1 usage
    """Open a CSV file and create a 2D list of it's contents"""
    file = [] # Empty 2D list

    # Attempt to open file
    try:
        with open(file_path, newline='') as csvfile:
            reader = csv.reader(csvfile)

            # If row contains number and name add to file
            for row in reader:
                if len(row) >= 2:
                    file.append(row)
    except FileNotFoundError:
        print("Error opening file...", end="")
        return[]

    # Collect course numbers from every row
    course_nums = {row[0] for row in file}

    # Check each row for prerequisites
    for row in file:
        if len(row) > 2:
            # Accept number and name as good
            good_fields = row[:2]

            # Check if prerequisite exists in course number collection
            for pre in row[2:]:
                # If exists, accept prerequisite as good, otherwise reject
                if pre in course_nums:
                    good_fields.append(pre)

            # Update row
            row[:] = good_fields

    return file
```

INSERT/ADD FUNCTIONS

```
def insert_course(self, course): 1 usage (1 dynamic)
    """Insert course at root or call add_course"""
    if self.root is None:
        self.root = Node(course)
    else:
        self.add_course(self.root, course)

def add_course(self, node, course): 3 usages
    """Recursively traverse tree and insert course"""
    if course.num < node.course.num:
        if node.left is None:
            # Add course to left child if empty
            node.left = Node(course)
        else:
            # Recursive call on left child
            self.add_course(node.left, course)
    else:
        if node.right is None:
            # Add course to right child if empty
            node.right = Node(course)
        else:
            # Recursive call on left child
            self.add_course(node.right, course)
```

2. Justify the inclusion of the artifact in your ePortfolio.

I selected this artifact because it contains advanced algorithmic concepts including recursion and parent/child nodes. I felt it would benefit from being translated to Python and having it's analysis functionality expanded. I have made the following improvements:

- The program was successfully translated to Python while preserving its functionality and BST. The Course structure was converted to a

LOAD/DISPLAY COURSES

Python @dataclass, and the BST, node structure, insert, search, traverse, CSV parse, course load, and main menu logic were all rewritten. The code was also streamlined with cleaner output, simplified input validation, and Pythonic file handling.

- New features were added such as a Find-by-Subject option and Subject sub-menu that allows users to view courses within a selected subject using the new search_subject and print_subject BST methods. A Statistics feature was also

introduced through the gen_stats and counter methods which print information such as the total number of courses, subject frequency, and most common prerequisites.

The Python version maintains the behavior of the original C++ while improving readability, expanding functionality, and making the program easier to maintain.

3. Did you meet the course outcomes you planned to meet with this enhancement?

The outcome I set out to meet with this category was: “Design and evaluate computing solutions that solve a given problem using algorithmic principles and computer science practices and standards appropriate to its solution, while managing the trade-offs involved in design choices.” I believe I’ve met this outcome because I’ve shown ability to write, comprehend, explain, and apply complex algorithms and data structures in a constructive way.

```
Main Menu
1. Load Courses
2. Display All Courses
3. Find A Course
4. Find A Subject
5. Course Statistics
6. Exit
Enter choice: 1
Enter CSV filepath: i1.csv

Loading CSV file...Done!

Main Menu
1. Load Courses
2. Display All Courses
3. Find A Course
4. Find A Subject
5. Course Statistics
6. Exit
Enter choice: 2
```

COURSE	PREREQUISITES
AE201 - Introduction to Aeronautics	None
AE301 - Flight Mechanics	AE201
BI0101 - General Biology	None
BI0201 - Genetics	BI0101
BI0301 - Microbiology	BI0201
BI0401 - Molecular Biology	BI0301 CH201
BI0402 - Advanced Genetics	BI0301
CE101 - Digital Logic	None
CE201 - Computer Organization	CE101
CE301 - Embedded Systems	CE201
CE401 - Microprocessor Design	CE301 EE201
CH101 - General Chemistry I	None
CH102 - General Chemistry II	CH101

FIND BY NUMBER/SEARCH SUBJECT

```
Main Menu
1. Load Courses
2. Display All Courses
3. Find A Course
4. Find A Subject
5. Course Statistics
6. Exit
Enter choice: 3
Enter course number: AE201

| COURSE                                     | PREREQUISITES
| AE201 - Introduction to Aeronautics       | None

Main Menu
1. Load Courses
2. Display All Courses
3. Find A Course
4. Find A Subject
5. Course Statistics
6. Exit
Enter choice: 4

Subject Menu
1. Search Subject By Abbreviation
2. Show Abbreviations
3. Back
Enter choice: 1
Enter subject abbreviation: AE

AE Courses:
AE201 - Introduction to Aeronautics
AE301 - Flight Mechanics
```

I finished by commenting the code thoroughly, adding a header, and making sure everything was formatted properly.

4. Reflect on the process of enhancing and modifying the artifact.

When updating this project I found that I hadn't worked in Python is quite some time. Once I was back in rhythm I was able to translate the existing codebase with little difficulty, even adding some formatting along the way.

STATS

When

adding functionality I really had to pay attention to datatypes and scope to make sure that I was working with fields when I wanted fields, and addresses when I wanted addresses. I would say the main challenge was:

- Adding the statistical option was challenging because I had to change how I approached it. The BST was a hand-written structure so it doesn't have built-in functions, this made taking statistics on its contents more interesting. The problem was solved with lists and a recursive algorithm that tracks courses and prerequisites that can then be analyzed.

```
Main Menu
1. Load Courses
2. Display All Courses
3. Find A Course
4. Find A Subject
5. Course Statistics
6. Exit
Enter choice: 5

TOTAL COURSES: 40

SUBJECT FREQUENCY
Computer Science      *****
Information Technology ****
Software Engineering ***
Computer Engineering ****
Electrical Engineering ****
Mechanical Engineering ****
Aerospace Engineering **
Chemistry             ****
Biology               *****

COMMON PREREQUISITES
BI0301: 2
CH201: 2
EE201: 2
CS102: 2
CS201: 2
CS301: 2
```